**Thapar Institute of Engineering and Technology**

Patiala, Punjab

UCS503P - Software Engineering

# ArogyaMitra

An AI-Assisted Real-Time Telemedicine Consultation Platform

Submitted to

**Dr. Paramveer Kaur**

Submitted by



| **Student** | **Roll Number** |
|---|---|
| Animesh Sudhanshu | 1024170375 |
| Ashish Bhagat | 1024170372 |
| Devansh Rathaur | 1024170380 |

Group: 3Q34 BE Third Year, CSE

Mid-Semester Evaluation

**Computer Science and Engineering Department**

# Contents

**1 Introduction** **1**

1.1 Background and Context . . . . . 1

1.2 Project Objectives . . . . . 2

1.3 Feasibility Study . . . . . 2

1.4 Scope and Deliverables . . . . . 2

**2 Methodology and Design** **4**

2.1 Requirements Engineering . . . . . 4

2.2 System Architecture . . . . . 5

2.3 UML Modelling . . . . . 6

2.4 Data Flow Diagrams . . . . . 9

2.5 SDLC Model Selection . . . . . 15

**3 Results and Evaluation** **16**

3.1 Testing Methodology . . . . . 16

3.2 Performance Metrics . . . . . 16

3.3 Analysis . . . . . 17

**4 Conclusion** **18**

4.1 Summary of Work . . . . . 18

4.2 Key Findings . . . . . 18

4.3 Future Work and Recommendations . . . . . 18

4.4 Final Remarks . . . . . 19

**References** **20**

# Chapter 1

# Introduction

## 1.1 Background and Context

ArogyaMitra is a web-based, AI-assisted real-time telemedicine platform. It reduces the friction between a patient who needs medical advice and a doctor who can give it, for non-emergency consultations, using chat, online booking and an AI assistant that prepares context before the doctor joins.

The broader motivation for the project is improving access to primary healthcare and healthcare equity. The immediate engineering objective is to translate low-friction, real-time doctor–patient consultation into a measurable, deployable software solution built and iterated on by a small student team. Real-time web technologies and affordable managed backends have matured enough that a small team can build a credible consultation platform in an academic timeframe, while conversational AI can plausibly assist (not replace) the pre-consultation triage step.

This is an engineering course project, so the focus is on integration, workflow correctness and safe deployment of existing, well-understood technologies rather than on novel algorithm research. Time-to-value is the primary heuristic: meaningful increments are delivered quickly by adapting an existing open-source reference implementation, prototyping the core consultation workflow and validating it within weekly iterations, with CI-backed automation introduced early to reduce release friction.

### 1.1.1 Problem Statement

Patients, particularly those in smaller towns with limited specialist availability, face delays and information loss when seeking timely medical advice for non-emergency conditions. The existing challenges are:

- **Access friction:** Booking an in-person visit means travel and waiting, even for simple, non-emergency queries.
- **Fragmented communication:** Follow-ups, advice and prescriptions are exchanged informally (calls, chat apps) with no persistent record.
- **No triage before consultation:** Patients cannot get a first-pass sense of urgency or prepare context before a doctor is available.
- **Scheduling opacity:** Patients cannot easily see doctor availability or track the status of a booked consultation.

## 1.2 Project Objectives

ArogyaMitra streamlines the consultation journey end to end through the following objectives:

1. **Patient portal and doctor dashboard:** Registration, profiles, doctor discovery, booking, consultation queue and history.
2. **Real-time chat consultation:** A secure chat room per consultation, joinable only in the booked time window.
3. **AI pre-consultation assistant:** Gemini-based symptom intake with an emergency-triage safety layer; the assistant never diagnoses or prescribes.
4. **Records and admin control:** Medical records and admin approval of doctors.

**Current status.** The frontend and backend currently run locally on the development machine (localhost) and have not yet been deployed to Netlify and Render. Online payments (Stripe) and video conferencing are not part of the current implementation and will be implemented in the future.

The **primary success metric** is a median time-to-connect (TTC) of at most 2 minutes between the scheduled start and both parties being in the same session.

## 1.3 Feasibility Study

Table 1.1 answers the six feasibility questions for the proposed platform.

| No. | Question | Answer |
|---|---|---|
| 1 | What if the system wasn't implemented? | Patients keep travelling and waiting for simple queries. Advice stays scattered over calls and chat apps with no record. |
| 2 | What are the current process problems? | No instant view of doctor availability or booking status. No triage or context before the consultation starts. |
| 3 | How will the proposed system help? | Remote booking, chat, persistent history and records. AI prepares symptoms for the doctor; emergencies are escalated. |
| 4 | What will be the integration problems? | Join control by booked hour. Clerk-JWT secured API; keeping AI vectors in sync via hooks. |
| 5 | Is new technology needed? Which skills? | No new hardware: Next.js, Node/Express, MongoDB. Skills: full-stack JavaScript, REST, prompt/tool design. |
| 6 | What facilities must be supported? | Managed hosting (Netlify, Render, planned; currently localhost) and MongoDB Atlas. SaaS: Clerk and Gemini; a browser. |

Table 1.1: Feasibility study of the ArogyaMitra platform

**Feasibility verdict:** Highly viable. The platform uses a standard managed stack, is built on a proven open-source codebase, and has minimal infrastructure demands.

## 1.4 Scope and Deliverables

### 1.4.1 Initial Deliverable (First Iteration)

- Fork/clone the reference repository and set up the local environment (done; the staging environment is planned).

- Patient and doctor authentication and profile pages.
- Appointment booking flow with doctor availability.
- Real-time chat consultation between doctor and patient.
- Basic logging and timestamps for TTC measurement.
- CI: build, lint and backend unit tests.
- CD to staging (Render and Netlify) with a single pipeline job (planned; the application currently runs on localhost).

### 1.4.2 Subsequent Deliverables

- Online payments (Stripe) and real-time video consultation (LiveKit): to be implemented in the future.
- Deployment of the frontend (Netlify) and backend (Render) from the current local setup.
- AI pre-consultation assistant with the emergency-triage safety layer.
- In-app alerts for upcoming consultations.
- Admin/analytics view of bookings, TTC and completion rate.
- Regression test suite for AI assistant safety behaviour.

# Chapter 2

# Methodology and Design

## 2.1 Requirements Engineering

### 2.1.1 Requirements Engineering Process

The requirements were developed through the seven standard requirements-engineering activities: (1) inception, (2) elicitation, (3) elaboration, (4) negotiation, (5) specification, (6) validation and (7) management.

### 2.1.2 Stakeholders

- **Primary actors:** Patient and Doctor (end users).
- **Secondary actor:** Administrator (approval and account management).
- **External systems:** Clerk and Gemini; Stripe and LiveKit will be added in the future.

### 2.1.3 Functional Requirements

| ID | Requirement | Description |
|---|---|---|
| FR1 | User authentication and roles | Clerk sign-in with JWT sessions; patient, doctor and admin roles. |
| FR2 | Doctor onboarding and approval | Doctors publish specialty, price and schedule; the admin approves or rejects. |
| FR3 | Discovery and booking | Patients search doctors and book a date and time slot. |
| FR4 | Payments | Online payments for paid doctors (Stripe Checkout) will be implemented in the future. |
| FR5 | Real-time consultation | Text chat in a per-consultation room, joinable only in the booked hour; video calls to be added in the future. |
| FR6 | Consultation lifecycle | pending → in-progress → completed / canceled; hourly auto-close. |
| FR7 | AI pre-consultation assistant | Symptom triage tools, emergency override, RAG over the patient's own records, PDF upload, hourly quota. |
| FR8 | Records and alerts | Consultation history, medical records and in-app consultation alerts. |

Table 2.1: Functional requirements

### 2.1.4 Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR1 | Performance | Median time-to-connect $\leq$ 2 min; streamed AI replies. |
| NFR2 | Security | Clerk JWT, role checks, participant check, helmet, input sanitising, rate limiting. |
| NFR3 | Usability | Responsive web UI for desktop, tablet and low-to-mid range phones. |
| NFR4 | Reliability | AI context search degrades gracefully. |
| NFR5 | Privacy and safety | Per-patient vector scope; the AI never diagnoses or prescribes. |

Table 2.2: Non-functional requirements

## 2.2 System Architecture

### 2.2.1 High-Level Design

ArogyaMitra follows a decoupled three-tier web architecture with external SaaS services (Figure 2.1). The client tier is a responsive web interface (desktop, tablet and phone browsers) with patient, doctor and admin portals. The presentation tier is a Next.js 16 / React 18 application, and the application tier is a Node.js/Express REST API (`/v1`). The target deployment is Netlify for the frontend and Render for the backend, but both currently run locally on the development machine (localhost). The data tier is MongoDB Atlas accessed through the Mongoose ODM. The tiers communicate over HTTPS, REST with a Clerk JWT, and Mongoose respectively. Clerk and the Google Gemini API are consumed as external services. Payment (Stripe) and video (LiveKit) services shown in the architecture figure are not part of the current implementation and will be integrated in the future.

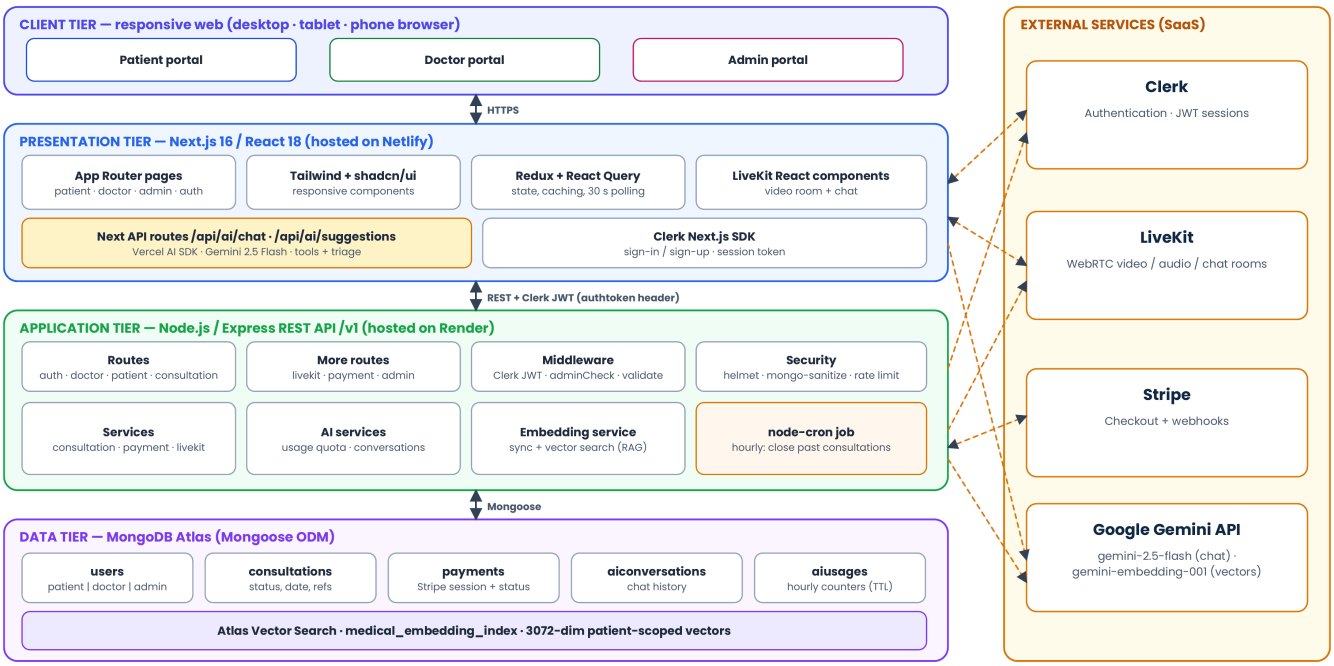



Figure 2.1: System architecture: three-tier web architecture with external SaaS services

### 2.2.2 Technical Stack

- **Framework/Language:** Next.js and React with Tailwind and shadcn/ui on the frontend [Ver26]; Node.js and Express on the backend [Ope26].
- **Database:** MongoDB Atlas with the Mongoose ODM and Atlas Vector Search for patient-scoped embeddings [Mon26].
- **Tools:** GitHub Actions for CI (build, lint and tests); Render (backend) and Netlify (frontend) for staging deployment, which is planned (the application currently runs on localhost).
- **Libraries and services:** Clerk for authentication and JWT sessions [Cle26]; Google Gemini (via the Vercel AI SDK) for the AI assistant [Goo26].
- **Planned for the future:** Stripe for online payments and LiveKit for video consultation.

### 2.2.3 AI Assistant Scope

The AI pre-consultation assistant is an integration task, not a research contribution:

- An existing LLM API (Gemini) is used for conversational symptom intake; there is no in-house model training.
- A lightweight safety/triage layer detects emergency-flagged symptoms (for example chest pain, stroke signs, self-harm) and always routes to a "seek immediate care" response rather than open-ended advice.
- The assistant only prepares context for the doctor; it never issues a diagnosis or prescription.

### 2.2.4 Scalability and Deployability

- **Theoretical scalability:** The frontend and backend are decoupled; REST endpoints are stateless; appointment and user queries are indexed in MongoDB.
- **Practical deployability:** The system is designed to run on common managed hosting: MongoDB Atlas as the database, Render and Netlify as deployment targets, and GitHub Actions for the staging/production pipeline. Deployment is still to be done; at present everything runs locally.

## 2.3 UML Modelling

The diagrams in this section describe the target design of the platform. Elements for online payments (Stripe) and video conferencing (LiveKit) that appear in them will be implemented in the future.

### 2.3.1 Use Case Model

The use case model (Figure 2.2) identifies the Patient, Doctor and Admin as human actors, and Clerk, Gemini and a cron scheduler as external systems in the current scope. The Stripe and LiveKit actors shown in the diagram stand for online payments and video conferencing, which will be implemented in the future.

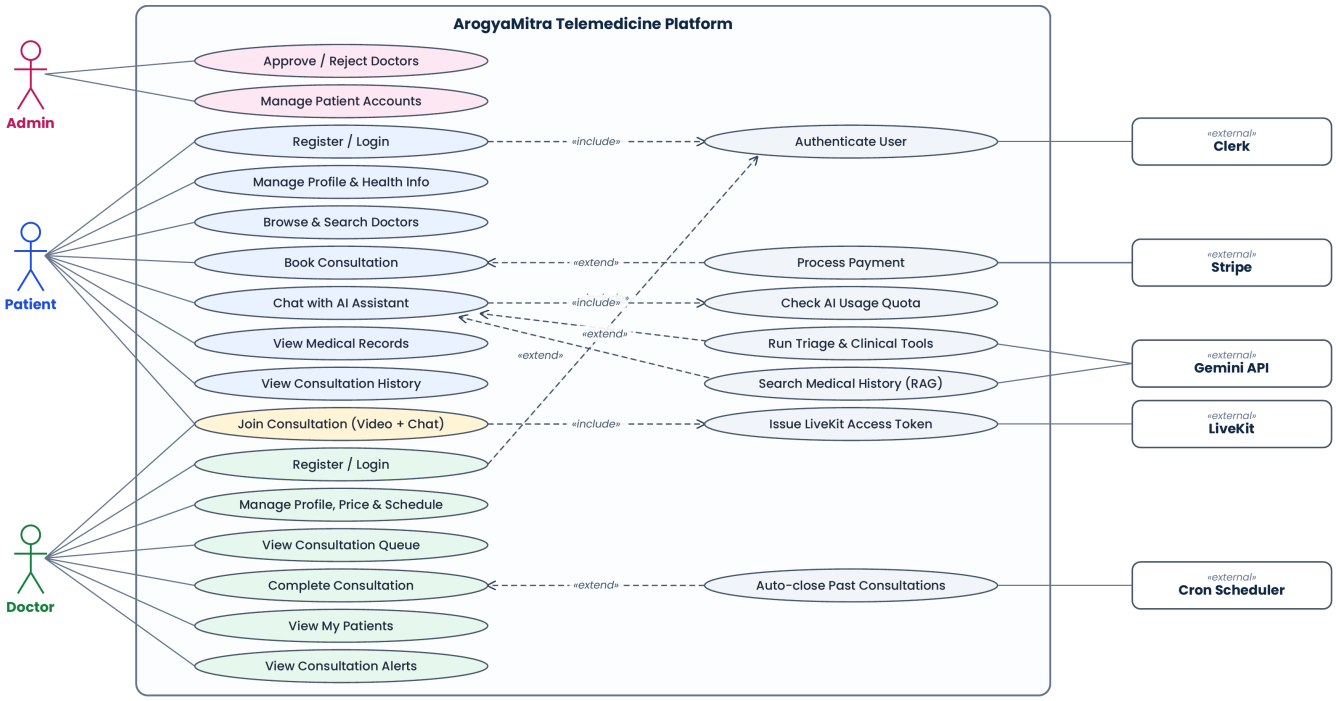



Figure 2.2: Use case diagram of the ArogyaMitra platform

### 2.3.2 Class Diagrams

The class diagrams are derived from the Mongoose models of the backend. Figure 2.3 shows the domain model (User and its Patient, Doctor and Admin specialisations, Consultation and Payment, and their enumerations). Figure 2.4 shows the AI assistant and medical-records classes (AiConversation, AiUsage, MedicalEmbedding and the RAG path).

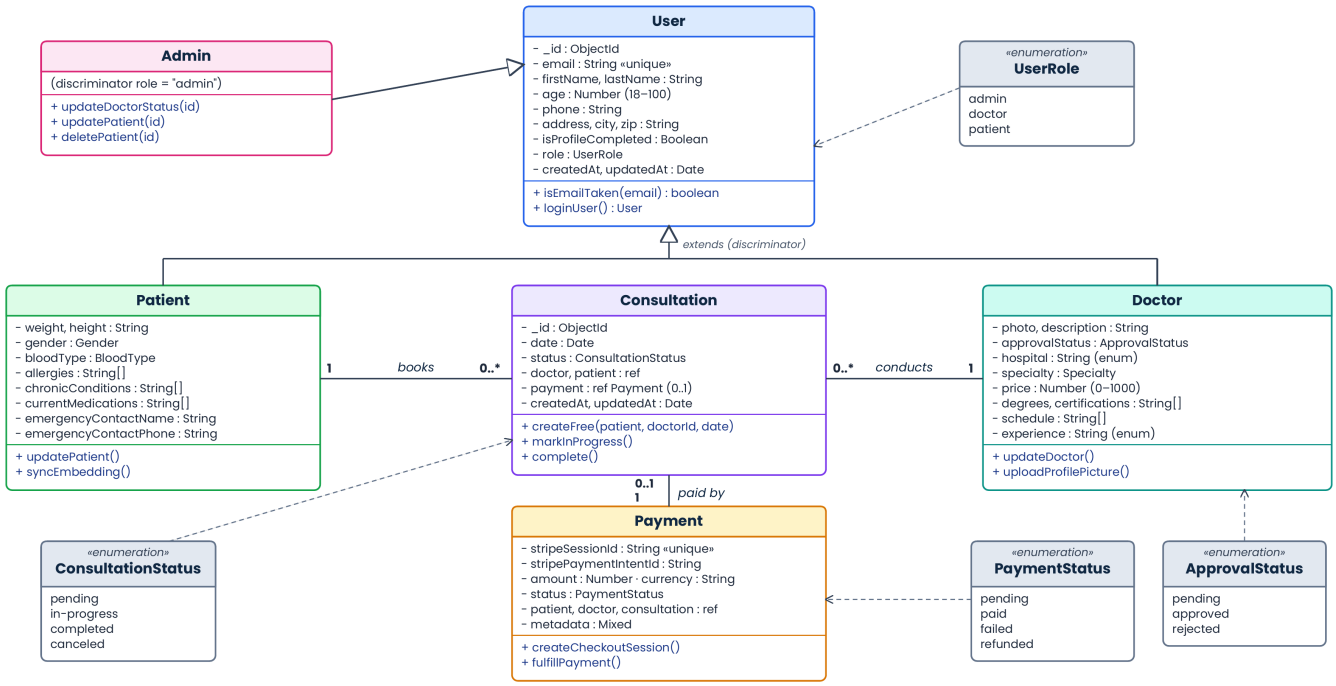



Figure 2.3: Class diagram: domain model

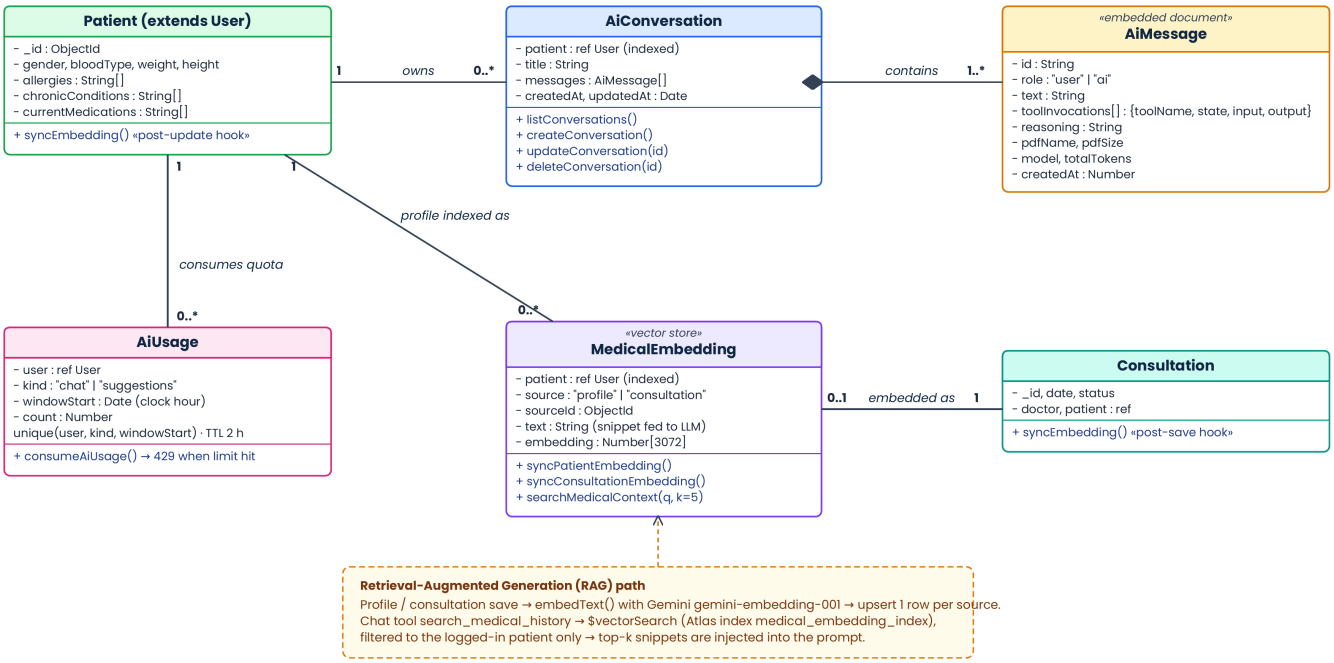



Figure 2.4: Class diagram: AI assistant and medical records (AiConversation, AiUsage, MedicalEmbedding, RAG path)

### 2.3.3 Activity Diagram

Figure 2.5 shows the end-to-end workflow, from sign-in to a completed consultation.

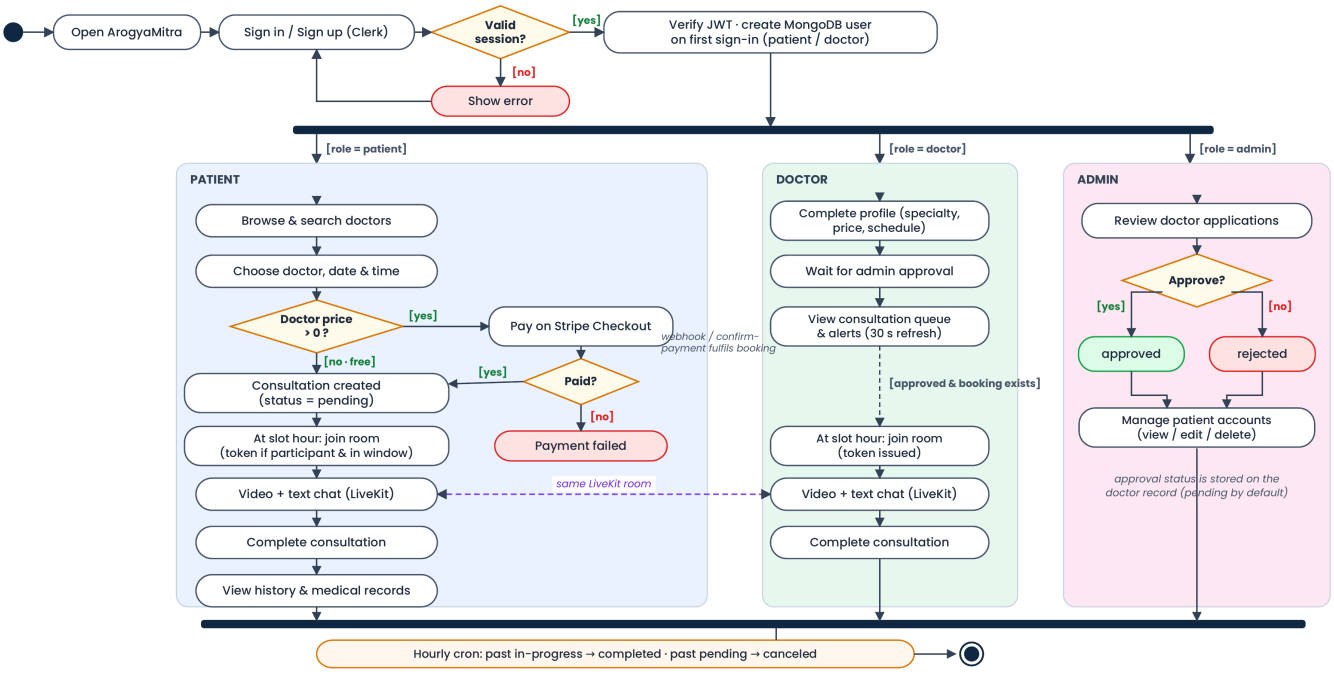



Figure 2.5: Activity diagram: end-to-end workflow from sign-in to completed consultation

### 2.3.4 Swimlane Diagram

Figure 2.6 shows who does what across the Patient, Platform, External services and Doctor lanes.

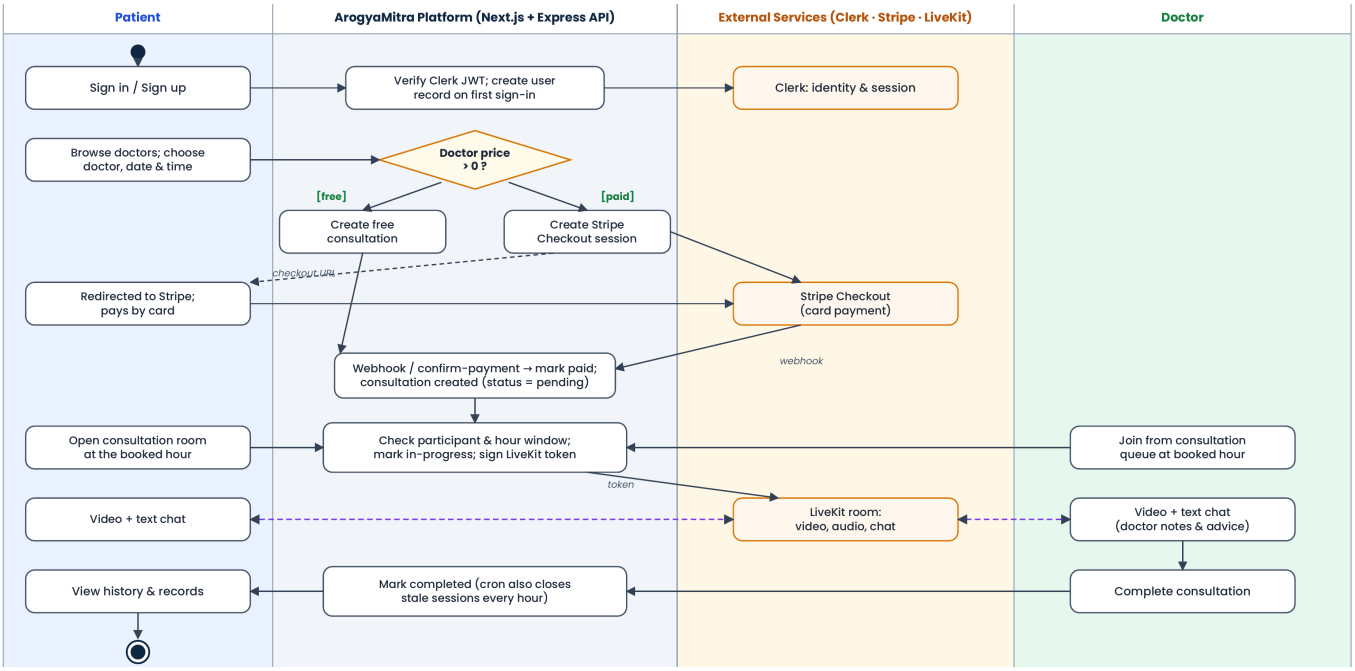



Figure 2.6: Swimlane diagram: Patient, Platform, External services and Doctor

### 2.3.5 State Diagrams

Figure 2.7 shows the Consultation, Payment and Doctor-approval lifecycles. A consultation is created as *pending*, becomes *in-progress* when a participant joins within the booked hour, and ends as *completed* or *canceled*; a cron job runs at minute 0 of every hour to close past consultations.

### 2.3.6 Sequence Diagrams

One sequence diagram describes the key AI interaction: the pre-consultation flow of quota check, Gemini tool calls, triage/RAG and persistence (Figure 2.8).

## 2.4 Data Flow Diagrams

The data flow diagrams use the following notation: a circle is a process, a rectangle is an external entity, and an open box is a data store. Figure 2.9 gives the Level 0 context diagram, Figure 2.10 the Level 1 decomposition, and Figures 2.11–2.16 the Level 2 diagrams of processes 1.0 to 6.0.

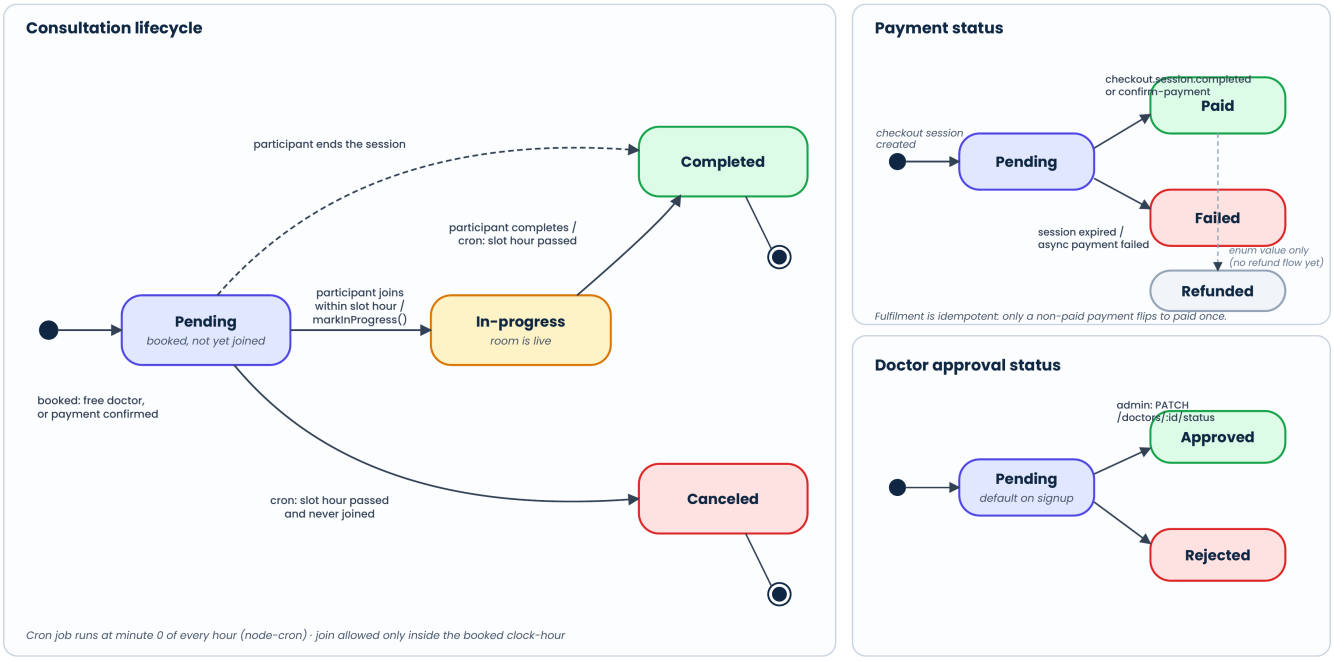



Figure 2.7: State diagrams: Consultation, Payment and Doctor-approval lifecycles

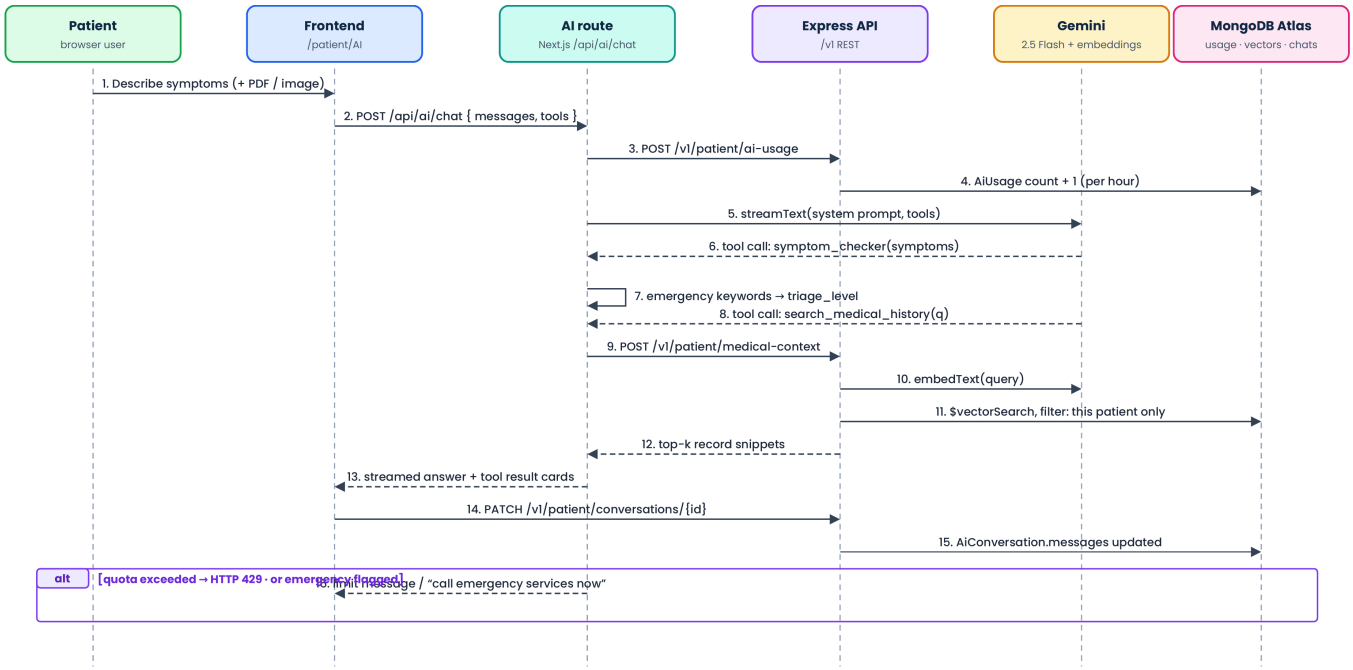



Figure 2.8: Sequence diagram: AI pre-consultation assistant (quota, Gemini tool calls, triage/RAG, persistence)

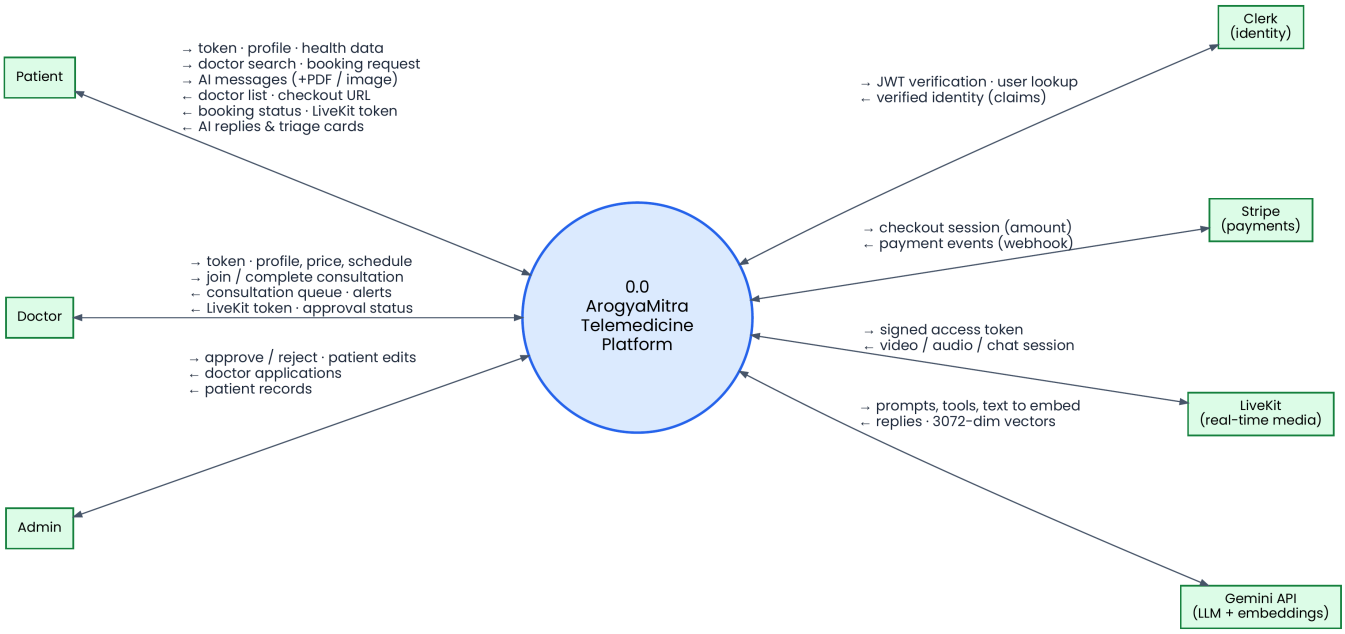



Figure 2.9: DFD Level 0: context diagram

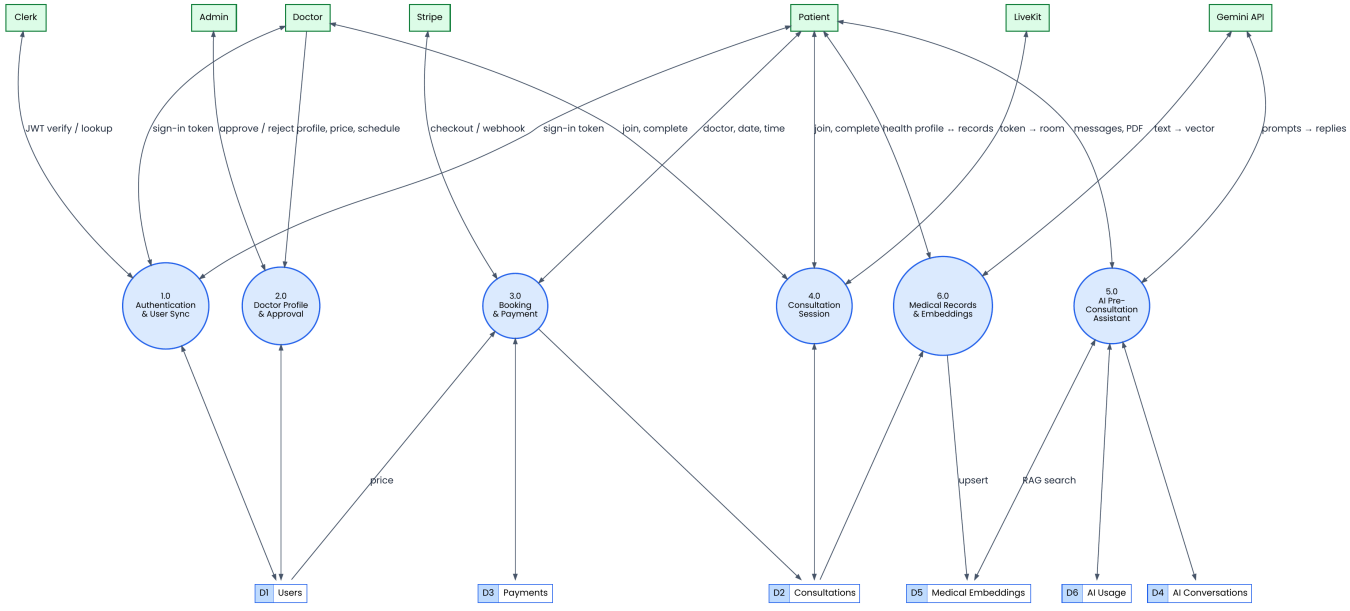



Figure 2.10: DFD Level 1

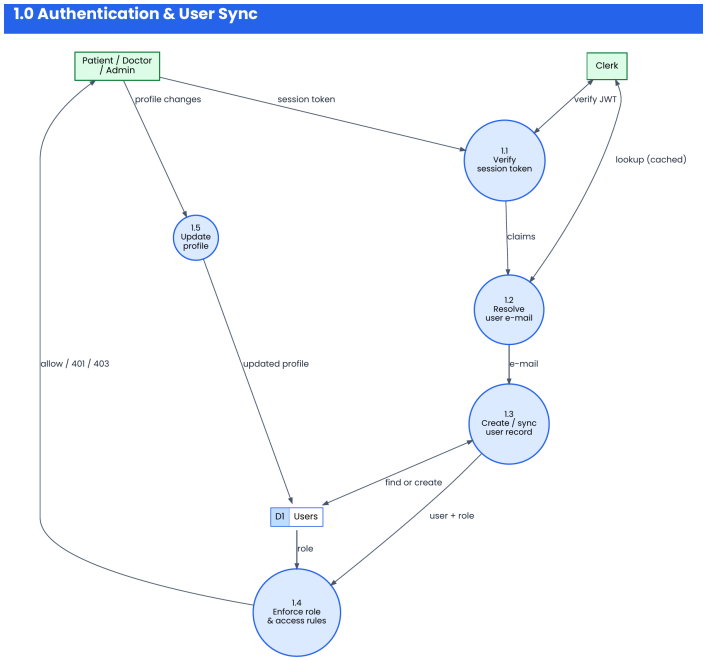



Figure 2.11: DFD Level 2: process 1.0, Authentication and user sync

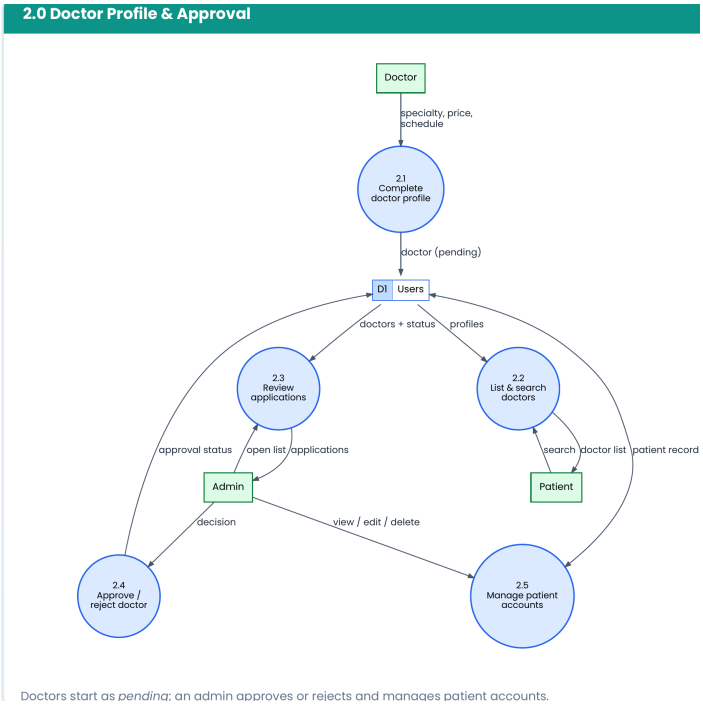



Figure 2.12: DFD Level 2: process 2.0, Doctor profile and approval

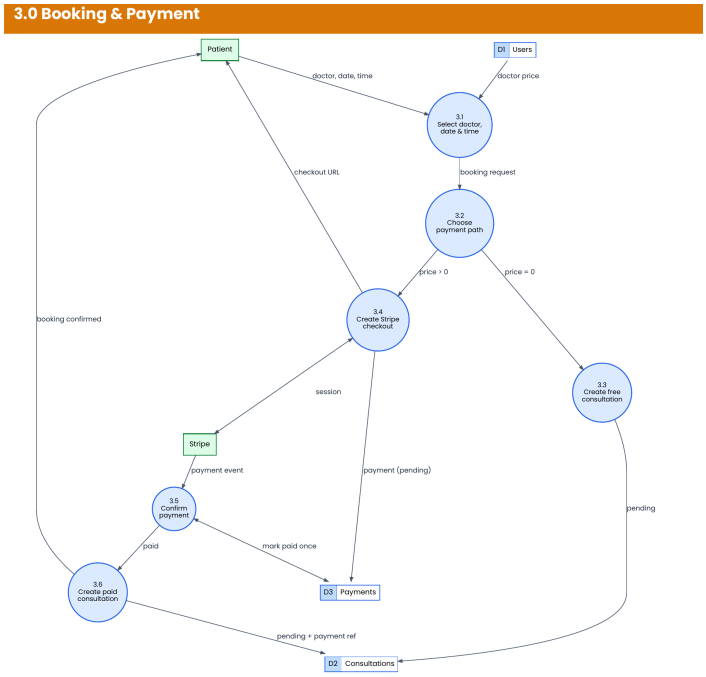



Figure 2.13: DFD Level 2: process 3.0, Booking and payment

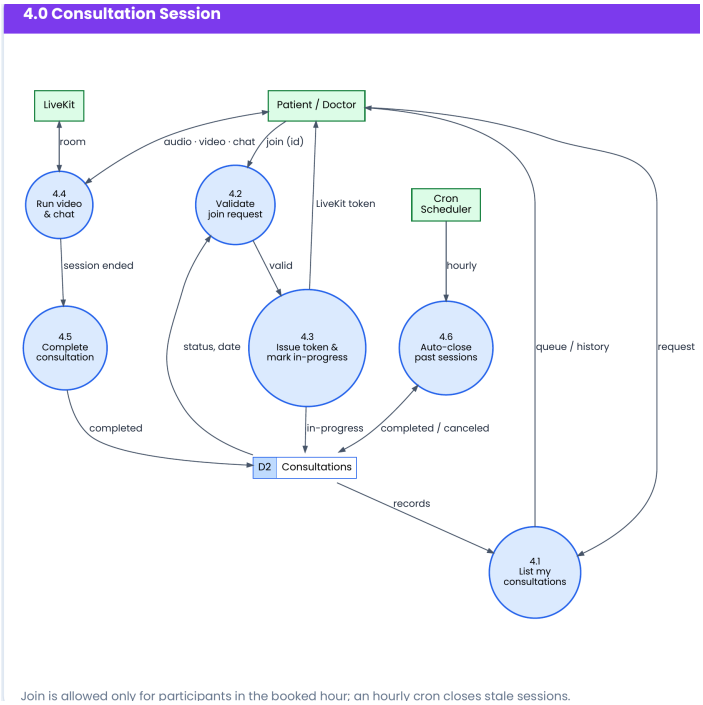



Figure 2.14: DFD Level 2: process 4.0, Consultation session

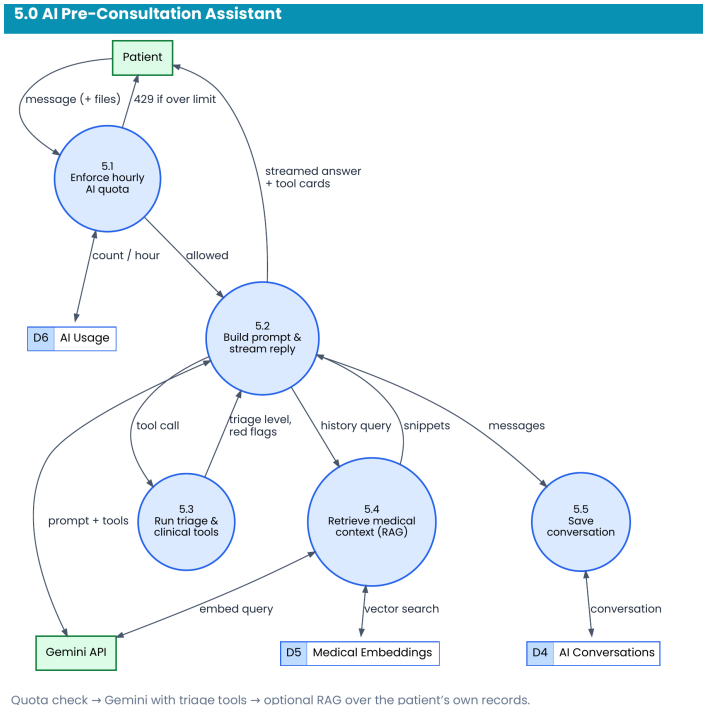



Figure 2.15: DFD Level 2: process 5.0, AI pre-consultation assistant

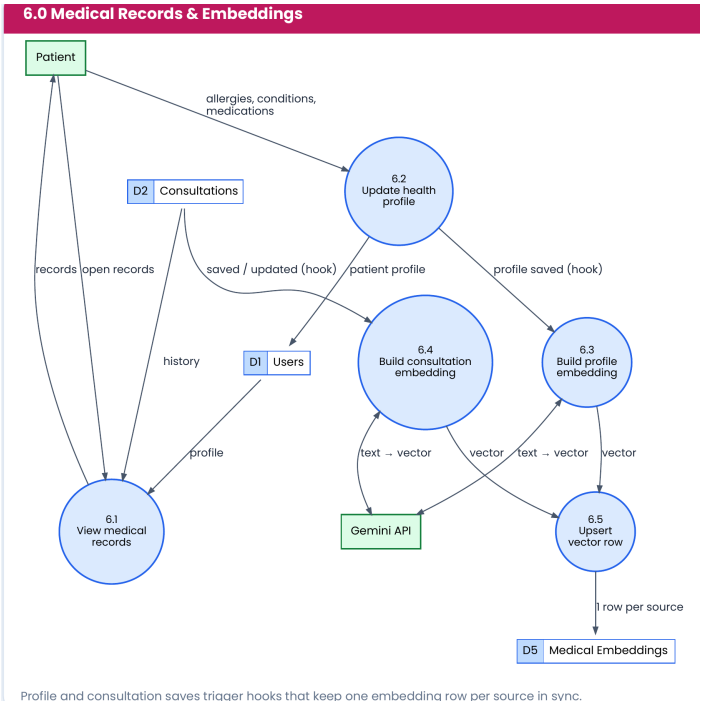



Figure 2.16: DFD Level 2: process 6.0, Medical records and embeddings

## 2.5 SDLC Model Selection

Waterfall, Prototyping and Agile Scrum were evaluated (Table 2.3); Agile Scrum was selected.

| Metric | Waterfall | Prototyping | Agile Scrum (selected) |
|---|---|---|---|
| Requirement adaptability | Rigid; changes are expensive once a phase concludes | Flexible for UI; weak for architectural evolution | High; requirements refined iteratively every sprint |
| Feedback frequency | Late; user evaluation only at final acceptance | Early UI feedback; variable back-end validation | Continuous; pilot users and supervisor review after each sprint |
| Risk and bug mitigation | Found late, during integrated testing | Medium; addressed during prototype refinement | Immediate; CI (build + lint + tests) on every change, AI safety regression suite |
| Delivery milestones | Single monolithic release at the end date | Demo prototype followed by a full build | Incremental; usable MVP first (auth + booking + chat), then AI and admin |

Table 2.3: Evaluation of SDLC models

- **How Scrum is applied:** Weekly iterations, each ending with a working increment. CI runs on GitHub Actions; staging on Render and Netlify is planned, since the application currently runs on localhost.
- **Release plan:** Sprint 1 delivers auth, profiles, booking, chat and TTC logging. Next come the AI assistant with safety tests, admin and deployment. Online payments and video consultation are planned for the future.
- **Why not the others:** Waterfall: AI and real-time behaviour need early feedback. Prototyping: weak for back-end integrity.

Agile Scrum supports iterative refinement, continuous feedback, early delivery and rapid risk mitigation.

# Chapter 3

# Results and Evaluation

## 3.1 Testing Methodology

The frontend and backend currently run on the local development machine (localhost) and have not yet been deployed to Netlify and Render. This chapter first records the current implementation status and then defines the evaluation plan against which the implementation will be measured.

### 3.1.1 Current Implementation Status

- **Environment:** Frontend and backend run locally on the development PC (localhost). Deployment to Netlify and Render is planned.
- **Payments and video:** The Stripe payment gateway and video conferencing are not implemented yet and will be implemented in the future.

### 3.1.2 Testing Plan

- **Unit Testing:** Backend unit tests run in CI together with the build and lint steps on every change.
- **Integration Testing:** Currently performed on localhost. Once deployed, repeatable deployments to the staging environment (Render and Netlify) via the CI/CD pipeline will exercise login, booking and consultation end to end.
- **Performance Testing:** Time-to-connect is measured from server-side timestamps (scheduled time versus session-join event).
- **User Testing:** A pilot with 10–15 patients and 3–5 volunteer or simulated doctor accounts, comparing TTC and booking completion rate across iterations.
- **AI Safety Testing:** A fixed regression suite of symptom prompts, including emergency cases, is run before each release to catch safety regressions.

## 3.2 Performance Metrics

Table 3.1 lists the metrics and targets used to evaluate ArogyaMitra.

| Metric | Target | How it is measured |
|---|---|---|
| Time-to-connect (primary) | Median $\leq$ 2 minutes | Server-side timestamps: scheduled start versus both parties in the same session |
| Booking completion rate | Compared across iterations | Share of started bookings that result in a confirmed appointment |
| AI assistant safety accuracy | Fixed suite must pass before every release | Share of emergency-flagged test cases routed to "seek immediate care" |
| User clarity | Simple 1–5 rating | Patient-reported clarity of the consultation flow |
| Reliability | e.g., $\geq$ 99% availability | Availability of login, booking and consultation on staging/production during test windows |

Table 3.1: Evaluation metrics and targets (to be measured during the pilot)

## 3.3 Analysis

- **Objectives:** Feasibility has been validated, the requirements are documented, and the UML and DFD models are complete. Online payments (Stripe), video conferencing and the Netlify/Render deployment are not yet implemented and will be done in the future; measured results will follow once they are done.
- **Challenges and mitigations:**
  - *Data privacy and consent:* collect only the medical data required for the consultation record, and do not persist raw AI conversations beyond what is needed for the doctor's context.
  - *AI safety failure:* a fixed emergency-prompt regression suite must pass before every release; the assistant never diagnoses or prescribes.
  - *Real-time reliability:* connection-state handling and reconnect logic so that a dropped session does not lose consultation context.
  - *Doctor adoption:* keep the doctor dashboard minimal to reduce onboarding friction during the pilot.
  - *Operational issues in the pilot:* deploy to staging early (the application currently runs only on localhost) and ensure CI/CD produces repeatable deployments before the pilot window.

# Chapter 4

# Conclusion

## 4.1 Summary of Work

This project proposes a deployable, AI-assisted real-time telemedicine platform that reduces the friction between patients and doctors for non-emergency consultations. It emphasises time-to-value through rapid iterations on a proven codebase, CI/CD for frequent safe releases, and measurable evaluation criteria (time-to-connect and AI assistant safety accuracy), staying focused on engineering integration and safe AI-assistance boundaries rather than research-driven novelty. At present the frontend and backend run locally on the development machine. Online payments (Stripe), video conferencing and the deployment to Netlify and Render are not part of the current implementation and will be implemented in the future.

## 4.2 Key Findings

- **Feasibility validated:** The platform is highly viable on a standard managed stack with minimal infrastructure demands.
- **SRS documented:** Eight functional and five non-functional requirements were specified.
- **UML and DFD modelled:** UML diagrams and DFD Levels 0–2 were produced.
- **Agile Scrum selected:** It best supports iterative refinement, continuous feedback, early delivery and rapid risk mitigation.

## 4.3 Future Work and Recommendations

1. Implement online payments with the Stripe payment gateway (checkout, webhook and idempotent fulfilment) in a future iteration.
2. Implement real-time video consultation (LiveKit) alongside the existing chat in a future iteration.
3. Deploy the frontend to Netlify and the backend to Render, with CI/CD to staging, replacing the current localhost setup.
4. Add the AI pre-consultation assistant with its emergency-triage regression suite and the admin/analytics view, then run the pilot and compare TTC and booking completion rate across iterations.

## 4.4 Final Remarks

The specification, models and process selected in this report give the team a clear and validated basis for building ArogyaMitra incrementally, with safety boundaries for the AI assistant defined from the start.

# Bibliography


[Cle26] Clerk, Inc. *Clerk Documentation*. Available at: `https://clerk.com/docs`. 2026 (cit. on p. 6).

[Goo26] Google. *Gemini API Documentation*. Available at: `https://ai.google.dev/gemini-api/docs`. 2026 (cit. on p. 6).

[Mon26] MongoDB, Inc. *MongoDB Atlas Documentation*. Available at: `https://www.mongodb.com/docs/atlas/`. 2026 (cit. on p. 6).

[Ope26] OpenJS Foundation. *Express: Fast, Unopinionated, Minimalist Web Framework for Node.js*. Available at: `https://expressjs.com/`. 2026 (cit. on p. 6).

[Ver26] Vercel. *Next.js Documentation*. Available at: `https://nextjs.org/docs`. 2026 (cit. on p. 6).